

User Knowledge Base for RAG

Purpose: A structured knowledge document containing the non-sensitive information available from the user's profile, prior learning context, projects, preferences, and goals. It is intended to be loaded into a vector database and retrieved through a RAG pipeline.

Important: This document is a synthesized knowledge base, not a verbatim transcript. It deliberately excludes sensitive personal data such as health conditions, political affiliation, religion, precise location, passwords, account identifiers, and similar information.

1. Professional Background

Role: SDET / Automation Tester.

The user has experience with software test automation and is actively expanding from traditional automation into modern web, mobile, CI/CD, AI application development, and AI testing/evaluation.

Programming languages: Java, JavaScript, TypeScript.

Core automation background: Appium with Java and Page Object Model (POM). The user is also working with WebdriverIO, Appium v2, TypeScript, Playwright, CI/CD, reporting, and test architecture.

2. Current Technical Direction

The user is moving toward a combined profile centered on advanced test automation plus AI testing/evaluation and AI application development.

- AI testing/evaluation: approximately 70% of the intended AI focus.
- AI development: approximately 30% of the intended AI focus.
- AI application topics: LLM applications, RAG pipelines, tool/function-calling systems, LangGraph workflows, MCP servers and clients, and Playwright MCP agents.
- AI testing topics: retrieval quality, hallucination, grounding, citations, latency, cost, prompt injection, and regression testing.

3. Web and Mobile Automation

WebdriverIO + Appium + TypeScript

The user has built or worked on a WDIO/Appium TypeScript framework with a layered architecture.

- Page Object Model (POM).
- BaseElement and BasePage abstractions.
- LoginPage and page-specific objects.
- Properties-file reader and environment configuration.
- Allure reporting.
- HooksManager with lifecycle hooks such as onPrepare, beforeTest, and afterTest.
- DriverManager layer.
- Automatic Appium service startup through WDIO.
- Smoke, regression, and sanity suite management.
- Automatic screenshot capture and attachment to Allure reports.
- Environment switching using TEST_ENV values such as QA/DEV/PROD.
- Platform switching for Android/iOS through capabilities.
- npm scripts for execution.
- Mocha-based test execution.

Playwright

The user is actively strengthening Playwright with JavaScript/TypeScript and wants deep practical and interview-level mastery.

- Playwright Test Runner.
- playwright.config.ts.
- Locators and locator strategy.
- Auto-waiting and actionability.
- Assertions and waiting mechanisms.
- Browser, BrowserContext, and Page architecture.
- Fixtures.
- Page Object Model and design patterns.
- Debugging and diagnostics.
- CI/CD integration.
- Reporting and analytics.
- Multi-environment configuration.
- Retries, workers, timeouts, screenshots, videos, and traces.
- Browser and device configuration, viewport, locale, and permissions.

4. Playwright Learning Path

The user's planned Playwright curriculum has been organized into phases and modules.

Phase 1 - Foundations

- Module 1: Playwright Fundamentals: what Playwright is, why it is used, Selenium vs Cypress vs Playwright, browser architecture, Browser → Context → Page, auto-waiting, actionability, internal execution flow, and design principles.
- Module 2: Installation and Project Setup: installation, browser installation, project structure, TypeScript setup, VS Code, Git, and Docker.
- Module 3: Configuration: playwright.config.ts, global/project configuration, browser/device configuration, viewport, locale, permissions, timeouts, retries, workers, screenshots, videos, traces, environment variables, and multi-environment configuration.

Module 4 - Locators

- Locator API and locator strategy.
- getByRole, getByText, getByLabel, getByPlaceholder, getByAltText, getByTitle, and getByTestId.
- CSS and XPath.
- Locator chaining.
- filter, has, hasText, nth, first, last, or, and.
- Frame locators.
- Shadow DOM.
- Dynamic elements.

Other planned modules

- Test Runner and Execution Control.
- Assertions and Waiting Mechanisms.
- Page Interactions.
- Browser Contexts and Storage Management.
- Test Fixtures.
- POM and Design Patterns.
- Debugging and Diagnostics.
- CI/CD Integration.
- Reporting and Analytics.

5. Playwright Practice Projects

E-commerce Practice Application

The user created a practice e-commerce application specifically to exercise Playwright automation.

- Built with Express and SQLite.
- Includes home, products, cart, and login pages.
- Used as a local/stage environment for automation practice.
- The user has worked on fixing broken image paths and improving the site's appearance.
- The user wants automation tests to run in response to a successful Vercel stage deployment/build.

GitHub Actions and Vercel

- GitHub repository used for the Playwright e-commerce practice project.
- Vercel is used for deployment.
- The user has worked on GitHub Actions workflows for Playwright.
- Playwright browsers are cached in CI.
- The user has practiced matrix-based shard execution.
- The user has practiced running Playwright shards across two machines.
- Blob reports are merged with Playwright's merge-reports workflow.
- The user has worked on balancing test execution using custom scripts based on file size/time.

6. Playwright Sharding and Reporting Knowledge

The user has practiced distributed Playwright execution using commands conceptually equivalent to `--shard=1/2` and `--shard=2/2`.

- Purpose of sharding: split test execution across workers/machines.
- Avoiding duplicate execution between shards.
- Generating blob reports per shard.
- Merging blob reports into a single Playwright report.
- Using GitHub Actions matrix jobs for shards.
- Caching Playwright browser binaries in CI.
- Experimenting with parser/balancer/executor scripts to distribute tests more evenly.

7. Performance Testing - k6

The user has been learning k6 and has focused on understanding the difference between metrics and workload concepts.

- vus and duration.
- thresholds.
- http_reqs versus iterations.
- Understanding what an iteration represents versus individual HTTP requests.
- Interpreting performance-test metrics rather than blindly reading numbers.

8. JavaScript and TypeScript Learning

The user is strengthening JavaScript fundamentals specifically to improve Playwright and automation skills.

- var, let, const.
- Hoisting.
- Temporal Dead Zone (TDZ).
- Loops.
- if/else and switch-case.
- Arrow functions.
- Arrays.
- map, filter, reduce.
- JavaScript/TypeScript interview questions for experienced SDETs.

9. LangChain and LLM Application Learning

The user has been learning LangChain concepts through hands-on code and is intentionally trying to understand execution flow rather than merely memorize APIs.

- LangChain chains and runnables.
- Runnable concepts.
- invoke and runnable execution.
- RunnableLambda.
- RunnableParallel.
- RunnableWithMessageHistory.
- SQLChatMessageHistory.
- ChatPromptTemplate.
- StrOutputParser.
- ChatOllama.
- dotenv / environment-variable configuration.
- Persistent chat-message history using SQL-backed storage.

The user has repeatedly focused on understanding what each runnable does, how inputs and outputs move through a chain, and how components compose together.

10. RAG Learning

The user is currently learning Retrieval-Augmented Generation (RAG) from the ground up.

- Core RAG idea: retrieve relevant information from an external knowledge source and provide it to an LLM as context.
- The user understands that sending a large amount of raw data directly to an LLM can reduce answer quality and increase the chance of irrelevant or hallucinated responses.
- The user has identified retrieval and generation as the major conceptual parts of a RAG pipeline.
- The user is learning document loading, chunking, embedding, vector storage, retrieval, and generation.
- The user is specifically experimenting with PDFs as the initial knowledge source.
- The user understands that RAG knowledge sources can also include structured and unstructured sources such as JSON, databases, websites, APIs, CSV files, and other documents.

PDF loading and chunking

The user has worked with PDF loaders such as PyPDF-based loaders and has been studying how documents are split into chunks.

The user specifically explored a scenario involving a one-page PDF, chunks, chunk size, and overlap. The conceptual goal is to make chunks small enough for useful retrieval while preserving enough neighboring context.

Conceptual overlap pattern: Chunk 2 contains some text from the end of Chunk 1 so that information split at a boundary is less likely to lose context.

Embeddings

The user has learned that text chunks are converted into numerical vector representations called embeddings. These vectors capture semantic characteristics of the text and allow similarity-based retrieval.

The user has specifically encountered the nomic-embed-text embedding model while using Ollama and has asked why model names appear in embedding code.

The user has also noticed that LLM model names such as Qwen can be passed to LLM components, and is distinguishing the role of a generation model from an embedding model.

Ollama and embedding batches

The user experienced an Ollama embedding failure when attempting to process all document chunks at once. The practical fix was to process a limited number of chunks per batch, such as eight chunks at a time, reducing the load on the local Ollama connection.

- PDFs are loaded.
- Documents are split into smaller chunks.
- Ollama creates numerical embeddings for chunks.
- Embeddings are generated in manageable batches.
- ChromaDB stores the vectors.
- A fresh ChromaDB persistence directory was used after the old database directory caused issues.

11. ChromaDB Learning

The user is actively learning ChromaDB as the vector datastore in a local RAG pipeline.

- Initializing a ChromaDB datastore.
- Persisting vector embeddings.
- Storing chunk content together with its vector representation and associated metadata.
- Retrieving relevant records from the vector dataset.
- Using the retrieved chunks as context for an LLM.

- Understanding the difference between storing vectors and retrieving semantically relevant text.

The user's current mental model is: Document → Load → Split into chunks → Embed chunks → Store embeddings in ChromaDB → Retrieve relevant chunks → Send retrieved context to LLM → Generate answer.

12. Ollama and Local AI Models

The user uses or experiments with Ollama for local LLM and embedding workflows.

- Ollama.
- nomic-embed-text for embeddings.
- Qwen-family models and other coding/LLM models.
- Phi-4.
- Code Llama.
- Qwen2.5-Coder.
- The user has also explored OpenRouter and other hosted/local model access concepts.

A key distinction the user is learning: an embedding model converts text into vectors for semantic search; a generative/chat model produces natural-language answers.

13. AI Agents, MCP, and Tool Use

The user's broader AI learning roadmap includes agentic systems and tool integration.

- AI agents.
- Tool/function calling.
- LangGraph workflows.
- MCP servers and clients.
- Playwright MCP agents.
- Jira MCP integration concepts.
- Using AI systems to fetch QA tickets and comments.
- Potential automated daily reporting workflows.

The user is interested in combining AI systems with practical software-testing workflows rather than studying LLMs only as theoretical concepts.

14. AI Testing and Evaluation Goals

The user wants to specialize in testing and evaluating AI systems, not only building them.

- Retrieval quality.
- Hallucination detection.
- Grounding evaluation.
- Citation correctness.
- Latency.
- Cost.
- Prompt injection and security testing.
- Regression testing for LLM applications.
- Evaluation of RAG retrieval and generation quality.
- Validation of tool/function-calling behavior.
- Testing agent workflows.

15. Teaching and Knowledge-Sharing Goals

The user also develops teaching material for Playwright and related automation topics.

- Wants beginner/intermediate-friendly Playwright training.
- Prefers practical HTML examples and hands-on scenarios.
- Wants lesson plans with concepts, API coverage, coding scenarios, interview questions, senior-level scenarios, and revision summaries.
- Has planned approximately 45 hours of Playwright training across around 30 modules.
- Tracks attendance and assignments.
- Uses Git repositories/folders for student assignments.
- Wants explanations that focus on practical understanding rather than buzzwords.

16. Learning Style and Working Preferences

- Prefers direct, practical explanations.
- Frequently asks for short answers when validating a single concept.
- For complex topics, prefers detailed step-by-step explanations and code-level reasoning.
- Often learns by writing or explaining code and then validating the mental model.
- Values understanding execution flow instead of memorizing syntax.
- Uses LinkedIn posts as a way to consolidate and communicate technical learning.
- Likes diagrams and simple visual explanations for concepts such as chunking, overlap, RAG flow, and framework architecture.
- Wants realistic interview questions suitable for an experienced SDET.

17. Career and Skill Development Direction

The user's skill-development direction can be summarized as:

- Strong software test automation foundation.
- Modern web automation with Playwright.
- Modern mobile automation with WDIO + Appium + TypeScript.
- CI/CD and distributed test execution.
- Performance testing with k6.
- JavaScript/TypeScript strengthening.
- LLM application development.
- RAG systems.
- Vector databases such as ChromaDB.
- Local models and Ollama.
- AI agents and MCP.
- AI testing and evaluation.

The intended differentiator is the combination of experienced SDET/automation knowledge with AI application testing and evaluation.

18. Known Practice Projects and Artifacts

- Playwright e-commerce practice application.
- WDIO + Appium + TypeScript mobile automation framework.
- GitHub Actions workflows for Playwright.
- Playwright sharding and blob-report merging experiments.
- k6 performance-testing exercises.
- LangChain runnable and message-history exercises.
- Local Ollama + ChromaDB RAG pipeline.
- PDF document ingestion and chunking experiments.
- LinkedIn technical-learning posts and diagrams.

19. RAG Retrieval-Friendly Facts

These compact facts are intentionally phrased so a vector search system can retrieve them from natural-language queries.

- User is an SDET with an automation-testing background.
- User has experience with Java, JavaScript, and TypeScript.
- User has worked with Appium Java and POM.
- User is learning WDIO, Appium v2, and TypeScript for mobile automation.
- User is learning and strengthening Playwright with JavaScript/TypeScript.
- User has experience with GitHub Actions and CI/CD.
- User has practiced Playwright sharding across multiple machines.
- User has practiced merging Playwright blob reports.
- User is learning k6 performance testing.
- User is learning LangChain chains and runnables.
- User is learning RunnableWithMessageHistory and SQLChatMessageHistory.
- User is learning RAG.
- User is learning PDF loading and chunking.
- User is learning embeddings.
- User has used Ollama for local model/embedding workflows.
- User has encountered nomic-embed-text as an embedding model.
- User is learning ChromaDB as a vector datastore.
- User is learning semantic retrieval from vector data.
- User is learning the difference between embedding models and generative LLMs.
- User is learning AI agents, MCP, LangGraph, and tool/function calling.
- User's AI focus is weighted toward AI testing/evaluation.
- User wants to test hallucination, grounding, retrieval, citations, latency, cost, injection, and regression.
- User creates Playwright training material and practical exercises.
- User prefers practical, direct, technically accurate explanations.

20. Suggested RAG Metadata

When chunking this document, useful metadata fields could include:

- topic: professional_background | playwright | wdio_appium | javascript | ci_cd | k6 | langchain | rag | embeddings | chromadb | ollama | agents | mcp | ai_testing | teaching
- source_type: user_knowledge_base
- knowledge_scope: non_sensitive_profile_and_learning_context
- content_version: 1
- retrieval_priority: high for technical-learning questions

21. Example Queries This Knowledge Base Should Answer

- What is my current AI learning focus?
- What RAG topics have I learned so far?
- Which embedding model have I used with Ollama?
- What vector database am I learning?
- What Playwright CI/CD topics have I practiced?
- What is my WDIO/Appium framework architecture?
- What LangChain runnable concepts have I studied?
- What is my AI testing specialization goal?
- What projects have I built for Playwright practice?
- What technologies am I combining for my career direction?
- What Playwright topics are in my learning syllabus?
- What do I know about sharding and blob reports?
- What is the difference between the embedding model and the LLM in my RAG setup?

22. Document Boundary

This knowledge base reflects information available in the assistant's current long-term conversation context and recent conversation summaries. It is not guaranteed to contain every fact ever shared by the user. It should therefore be treated as a practical RAG test dataset representing the user's known technical-learning context.

For RAG experiments, this document is especially suitable for testing semantic retrieval because it contains overlapping concepts across Playwright, SDET work, AI, RAG, LangChain, Ollama, embeddings, ChromaDB, and CI/CD.